

Welcome to 151B: Deep Learning

Instructor: Annapurna Vadaparty

TA: Ben Bao

Tutors: Alexander Radulescu, Trevor Duong

Course questions?

- PL questions
 - Might have errors/watermarks. Let us know about any issues, and if there are recurring issues that aren't fixed by tonight we might drop them
- Practice exam 1 is up (was posted on Sunday/mentioned on Piazza)
- Questions about what's on the exam? How are we feeling about it?
 - Any topics you're unsure about? Know where to find everything?
- Make sure you've scheduled in the CBTF/TTC for Thurs/Fri (and make sure if you chose Thurs that it doesn't conflict with your oral assessment time)
- LPRR A0 Updates: Jul 10

Some problems we encounter with generalization

- Overfitting
 - (Prior ML classes—we'll talk about solutions but hopefully you already know what this is)
- Underfitting
 - (Prior ML classes—we'll talk about solutions but hopefully you already know what this is)
- Vanishing gradient
 - We encounter this specifically in deep learning when we have many many layers of neural units
 - What is the vanishing gradient problem?

Today we'll be walking through several strategies and “Tricks of the Trade” to mitigate these problems.

A few strategies for overfitting

- The best way: Get more data! (Or, manufacture more data...)--this is what all those transformations in PyTorch do.
- Minimize $J=E+\lambda C$ where E is the error and C is a measure of model Complexity (regularization).

≡ Occam's razor

Article [Talk](#)

From Wikipedia, the free encyclopedia

For other uses, see [Occam's Razor \(disambiguation\)](#).

In [philosophy](#), **Occam's razor** (also spelled **Ockham's razor** or **Ocham's razor**; [Latin](#): *novacula Occami*) is the [problem-solving](#) principle that recommends searching for explanations constructed with the smallest possible set of elements. It is also known as the **principle of parsimony** or the **law of parsimony** ([Latin](#): *lex parsimoniae*). Attributed to [William of Ockham](#), a 14th-century English [philosopher](#) and [theologian](#), it is frequently cited as *Entia non sunt multiplicanda praeter necessitatem*, which translates as "Entities must not be multiplied beyond necessity",^{[1][2]} although Occam never used those exact words. Popularly, the principle is sometimes paraphrased as "of two competing theories, the simpler explanation of an entity is to be preferred".^[3]

How do we choose complexity measure C in $J=E+\lambda C$?

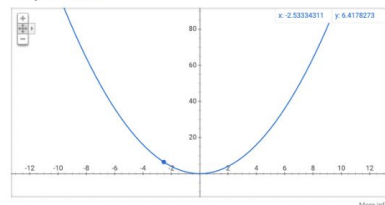
General Idea: Ockham's Razor: Make the model “smaller”

- L2 regularization: Minimize $\|w\|_2^2$
 - Derivative for each weight: $2w$
 - Make the weight smaller in proportion to its size
- L1 regularization: Minimize $|w|$
 - Derivative: 1
 - Make the weight smaller at a constant rate

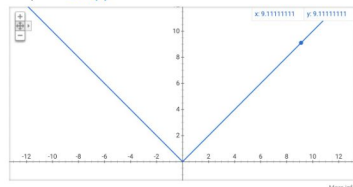
How do we choose complexity measure C in $J=E+\lambda C$?

- L2 regularization: Minimize $\|w\|_2^2$
 - Big weights more: the slope (how much J goes up for a given weight) is higher for larger weights
- L1 regularization: Minimize $|w|$
 - All weights the same : the slope (how much J goes up for a given weight) is constant
- Rumelhart's idea: Minimize $C = \|w\|^2 / (\|w\|^2 + 1)$
 - What does this do? (5 mins)

Graph for x^2



Graph for $\text{abs}(x)$



What is your competitive advantage? Why this class?

≡ Competitive advantage

Article [Talk](#)

From Wikipedia, the free encyclopedia

Not to be confused with [Absolute advantage](#) or [Comparative advantage](#).

In business, a **competitive advantage** is an attribute that allows an organization to outperform its [competitors](#).

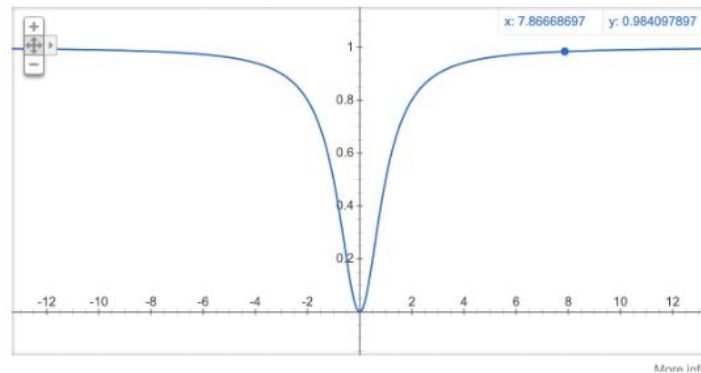
A competitive advantage may include access to [natural resources](#), such as high-grade ores or a low-cost power source, highly skilled labor, geographic location, high entry barriers, and access to new technology and to proprietary information.

You, with your knowledge of how models work, are equipped to reason through new loss functions in new situations.

Rumelhart's idea: Minimize $C = ||w||^2 / (||w||^2 + 1)$

- Rumelhart's idea: Minimize $C = ||w||^2 / (||w||^2 + 1)$
 - What does this do?

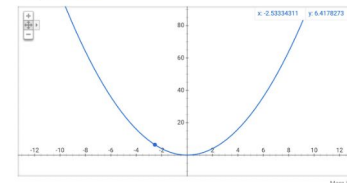
Graph for $x^2/(x^2+1)$



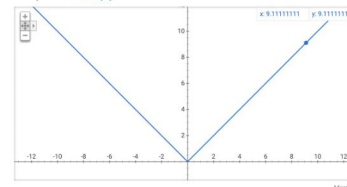
How do we choose complexity measure C in $J=E+\lambda C$?

- L2 regularization: Minimize $\|w\|_2^2$
 - Big weights more
- L1 regularization: Minimize $|w|$
 - All weights the same
- Rumelhart's idea: Minimize $C = \|w\|^2 / (\|w\|^2 + 1)$
 - Eliminate small weights (penalizes big weights less while penalizing small weights more, driving them to 0.)

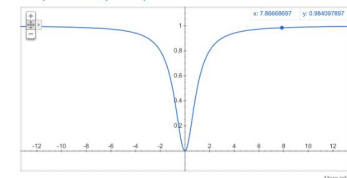
Graph for x^2



Graph for $\text{abs}(x)$



Graph for $x^2/(x^2+1)$



Another idea: Dropout

- For any hidden layer, probabilistically turn off some fraction of the hidden units
- This makes it so that no hidden unit can depend on any other
- When would you want to do this? Not want to do this?
 - (Maybe an example from yesterday?)

Dropout: exponential number of models

- Explores an exponential number of models
- Assume dropout rate is 0.5
- What happens if we have 4 hidden units?

Can have (Assuming exactly half turn off):

- 0011
- 1001
- 1010
- 1100
- 0101
- 0110
- This is 4 choose 2 (6)

Dropout: exponential number of models

- 4 choose 2: 6
- 8 choose 4: 70
- 10 choose 5: 252
- 12 choose 6: 924
- 100 choose 50: e^{29}
- So if exactly half turn off, we won't actually investigate an exponential number of models (unless we train for e^{29} epochs...) but we'll try.

Another Strategy: Early stopping

- Have a hold out set (some fraction of the training set) – this is a stand-in for the unseen test set
- Use the remaining portion of the training set to change the weights
- Watch the error on the holdout set and stop when it starts to rise

Another strategy: Add noise to...

- Where are some places we could add noise that you think would help with generalization?

Another strategy: Add noise to the training set or the model

- Add a small amount of Gaussian random noise to the inputs
- Add a small amount of Gaussian random noise to the hidden unit activations
- Both make the model more robust to perturbations: makes them generalize better.

So: to improve generalization (summary)

- Get more data or create more data artificially
- Complexity minimization (regularization)
- Dropout
- Early stopping
- Add noise to the Input or the model or the output!

Break!

See you in 5 mins.

“Tricks of the Trade”: More tomorrow, see [link](#) on course schedule

- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs
- Change the sigmoid
- Initialize the weights
- Momentum
- Adaptive learning rates

Questions, homework, and exam study time

End of where we reached in class

Follow-Ups/Corrections

- None for today